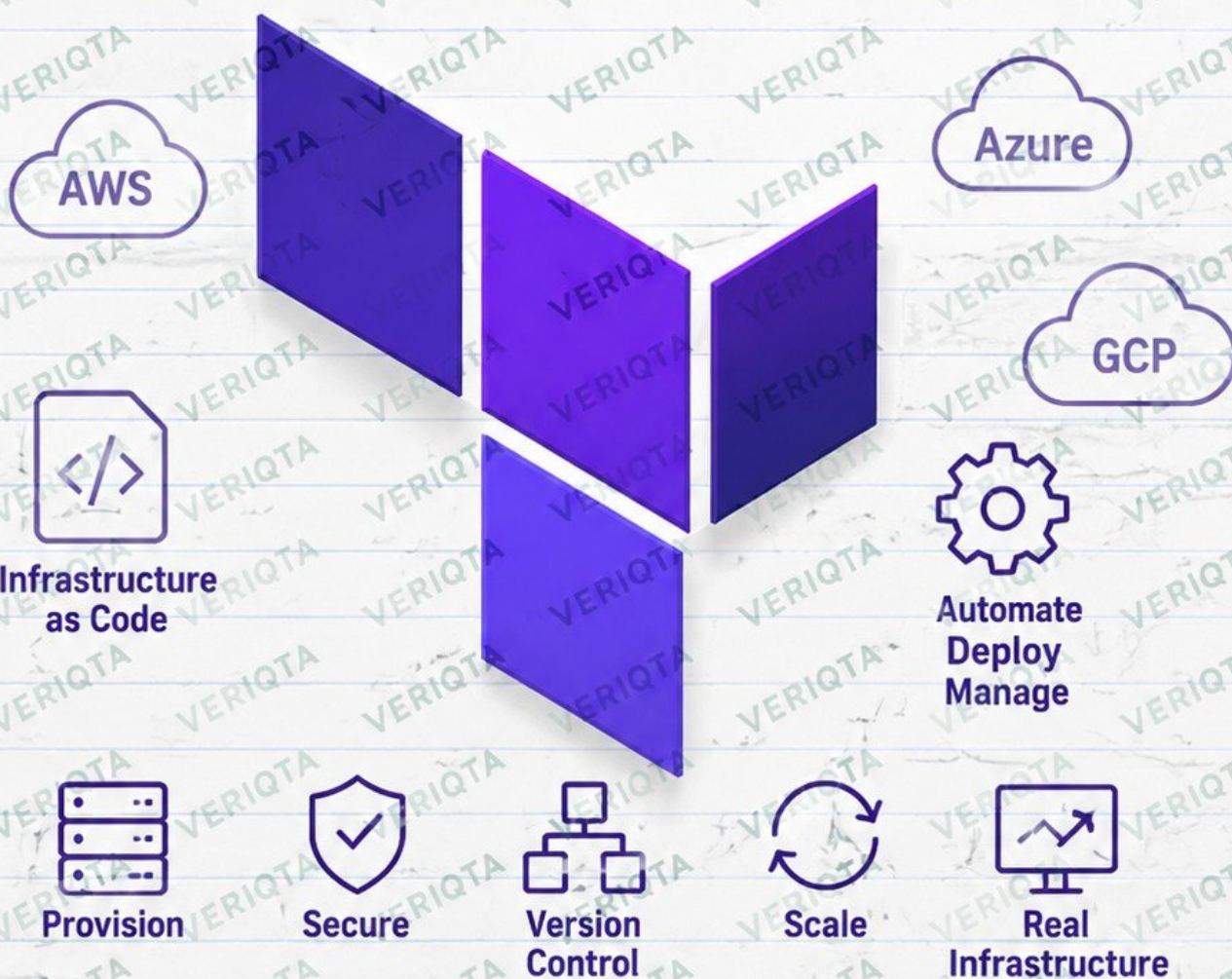


Terraform

CRASH COURSE



FROM ZERO TO PRODUCTION
BUILD • DEPLOY • MANAGE • AUTOMATE



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



01. WHAT IS TERRAFORM?

Page 01/20



WHAT IS TERRAFORM?

- Terraform is an open source tool from HashiCorp used to create, update and manage **infrastructure as code**.
- You write **configuration files**, Terraform talks to cloud provider APIs and provisions the resources for you.

WHY DOES IT EXIST?

- Provisioning infrastructure manually is **slow, error prone and hard to repeat**.
- Terraform was created to solve this by **using code** to define and manage infrastructure.
- It gives engineers a **consistent, repeatable and version controlled** way to build infrastructure.

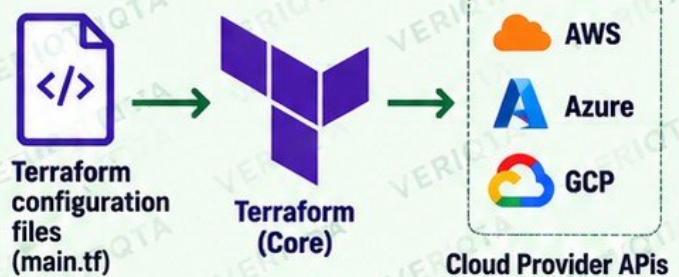
MANUAL vs AUTOMATED PROVISIONING

MANUAL PROVISIONING



- ✗ Time consuming
- ✗ Prone to human error
- ✗ Not consistent
- ✗ Difficult to repeat
- ✗ Hard to manage changes

TERRAFORM (AUTOMATED)



- ✓ Fast and automated
- ✓ Consistent and repeatable
- ✓ Version controlled (Git)
- ✓ Reduces human error
- ✓ Easy to make and track changes

KEY CONCEPT: INFRASTRUCTURE AS CODE (IaC)

- Infrastructure as Code (IaC) means you define your infrastructure using code instead of manual steps.
- Your configuration files describe the **desired state** of your infrastructure.
- Terraform figures out what needs to be **created, updated or deleted** to match that state.

IMPORTANT POINTS

- Terraform works with many providers (AWS, Azure, GCP and more).
- You write code in **HashiCorp Configuration Language (HCL)**.
- Terraform keeps track of what it created using a **state file**.
- You can review changes before they are applied using a **plan**.

REAL DEVOPS USE CASES

- Provisioning cloud infrastructure (**VPC, EC2, RDS, IAM, networking**).
- Creating consistent **development, staging and production environments**.
- Managing infrastructure for applications in **Kubernetes**.
- Building reusable infrastructure modules.
- Using Terraform in **CI/CD** pipelines.
- **Multi cloud** infrastructure management.



JUNIOR ENGINEER TIP

Think of Terraform as the code version of the cloud console.
You describe what you want, and it builds it for you consistently.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



02. HOW TERRAFORM WORKS

Page 02/20



WHAT HAPPENS?

- You write Terraform **configuration files**.
- Terraform reads the files and builds an **execution plan**.
- It uses **providers** to communicate with cloud or infrastructure **APIs**.
- It **creates, updates or deletes** resources to match your configuration.

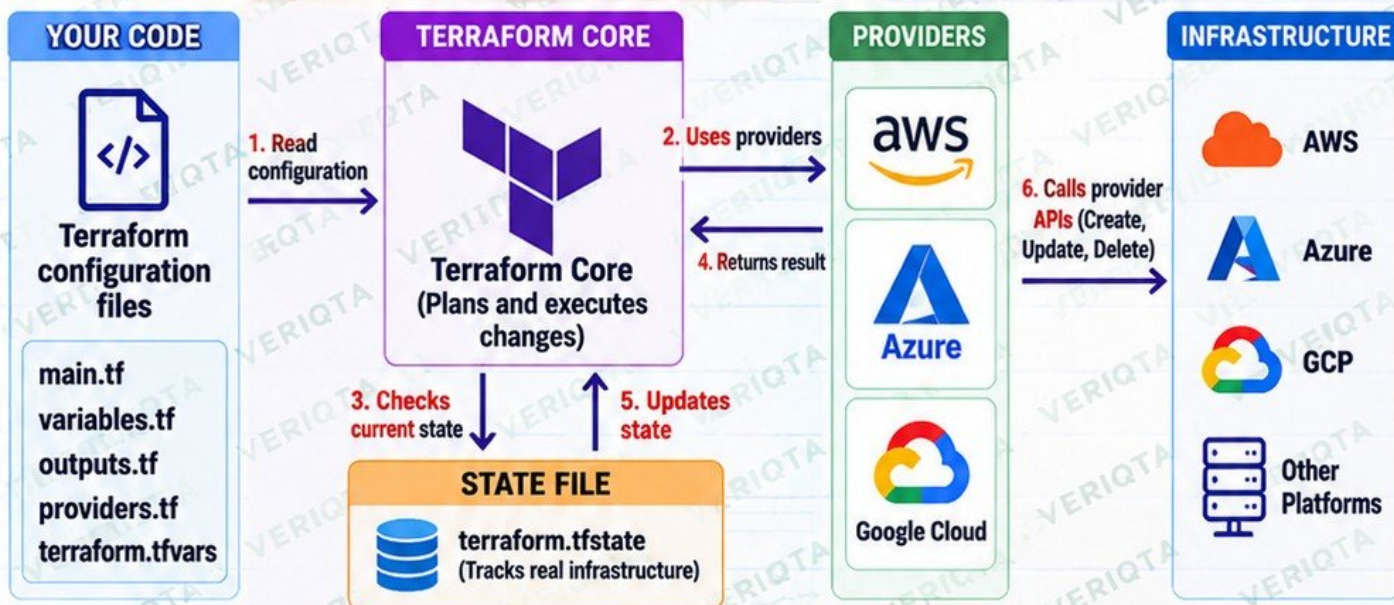
KEY COMPONENTS

- **Configuration Files** (What you want)
- **Terraform Core** (Processes your configuration)
- **Providers** (Plugins that talk to APIs)
- **State** (Tracks real infrastructure)
- **Infrastructure APIs** (Cloud providers / platforms)

WHY THIS DESIGN?

- Providers make Terraform work with **many platforms** (AWS, Azure, GCP and more).
- Configuration files define the **desired state**.
- Terraform Core compares desired state vs **current state**.
- State keeps track of what exists so Terraform **knows what to change**.

TERRAFORM ARCHITECTURE (SIMPLE VIEW)



PROVISIONING WORKFLOW

- 1 **terraform init**
Downloads required providers.
- 2 **terraform plan**
Shows what will be created, updated or deleted.
- 3 **terraform apply**
Executes the plan and provisions resources.
- 4 **terraform destroy**
Deletes the resources (when needed).

HOW TERRAFORM KNOWS WHAT TO DO

- Terraform reads the configuration files.
- It checks the current state (terraform.tfstate).
- It compares the **desired state vs current state**.
- It creates a plan of the changes required.
- It uses the correct provider to call the platform API.
- After successful provisioning, it **updates the state file**.

EXAMPLE FLOW

- You define an AWS EC2 instance in **main.tf**.
- Terraform checks the state (does it already exist?).
- If not, it uses the AWS provider to call the **AWS API**.
- AWS creates the EC2 instance.
- Terraform updates the **state file** with the new resource details.



JUNIOR ENGINEER TIP

Terraform does not create infrastructure randomly.
It follows your configuration and uses state to keep everything in sync.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



03. INSTALLING TERRAFORM

Page 03/20



WHAT YOU WILL DO

- Install Terraform on Linux, macOS and Windows.
- Verify the installation and version.
- Learn basic CLI commands.
- Create a project directory for your Terraform work.

PREREQUISITES

- A computer with internet access.
- A terminal (Linux/macOS) or Command Prompt/PowerShell (Windows).
- Basic understanding of directories.
- Administrator or sudo permission (may be required on some systems).

INSTALL ON LINUX (Ubuntu/Debian)

- 1 Update package list
`sudo apt update`
- 2 Install required tools
`sudo apt install -y wget unzip`
- 3 Download Terraform (example version)
`wget https://releases.hashicorp.com/terraform/1.6.6/terraform_1.6.6_linux_amd64.zip`
- 4 Unzip and move to system path
`unzip terraform_1.6.6_linux_amd64.zip
sudo mv terraform /usr/local/bin/`
- 5 Verify installation
`terraform version`

INSTALL ON macOS (using Homebrew)

- 1 Install using Homebrew
`brew install terraform`
- 2 Verify installation
`terraform version`

✓ Homebrew not installed?

Install Homebrew first:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

INSTALL ON WINDOWS (using Chocolatey)

- 1 Open PowerShell (as Administrator)
`choco install terraform`
- 2 Verify installation
`terraform version`

✓ Alternative: Manual Install

1. Download from official site.
2. Unzip the file.
3. Move terraform.exe to a folder in your system PATH.
4. Verify with:

```
terraform version
```

CHECK YOUR VERSION

```
terraform version
Terraform v1.6.6
on linux_amd64
+ provider registry.terraform.io/hashicorp/aws v5.28.0
```

- You should see the Terraform version.
- It confirms the installation was successful.
- The output also shows your operating system and installed provider plugins (if any).

BASIC CLI COMMANDS

COMMAND	WHAT IT DOES
<code>terraform version</code>	Shows installed version.
<code>terraform init</code>	Downloads required providers.
<code>terraform fmt</code>	Formats your code.
<code>terraform validate</code>	Checks for configuration errors.
<code>terraform plan</code>	Shows what will be created or changed.
<code>terraform apply</code>	Creates or updates resources.
<code>terraform destroy</code>	Deletes the resources.

CREATE A PROJECT DIRECTORY

- 1 Create a folder for your project
`mkdir terraform-project`
- 2 Move into the folder
`cd terraform-project`
- 3 Create a main.tf file
`touch main.tf`

✓ You are now ready to start writing your Terraform configuration files.



JUNIOR ENGINEER TIP

Keep your Terraform version up to date to get new features, security fixes and provider improvements.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



04. TERRAFORM PROJECT STRUCTURE

Page 04/20



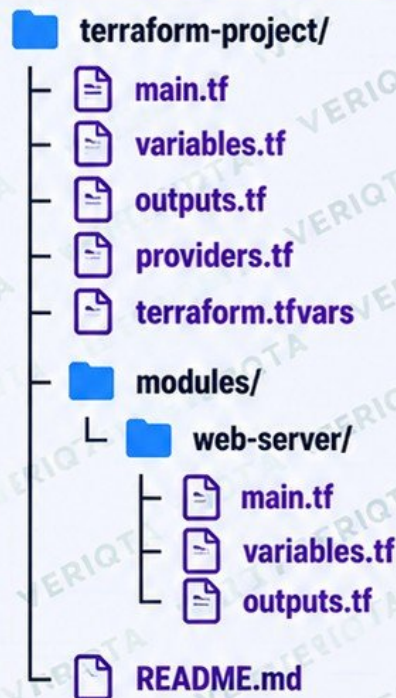
WHAT IS A TERRAFORM PROJECT STRUCTURE?

- A Terraform project is a set of **configuration files** that define your **infrastructure**.
- Each file has a **specific purpose** (provider, resources, variables, outputs, etc).
- This structure keeps your code **organized, readable and easy to maintain**.
- You can start simple and add **more files as your project grows**.

BENEFITS

- Organized and easy to understand
- Reusable and maintainable
- Supports team collaboration
- Makes it easier to manage complex infrastructure
- Follows Terraform best practices

PROJECT DIRECTORY EXAMPLE



main.tf

- Defines your **main resources** and infrastructure.
- Tells Terraform what to create, update or manage.

```
resource "aws_instance" "web" {
  ami      = "ami-1234567890abcdef"
  instance_type = "t2.micro"
}
```

KEY POINTS

- Contains resource blocks
- Uses provider resources
- Can include multiple resources
- Represents your desired state

variables.tf

- Declares input **variables**.
- Allows you to customize values without changing the main code.

```
variable "instance_type" {
  type        = string
  default     = "t2.micro"
  description = "EC2 instance type"
}
```

KEY POINTS

- Defines variable name, type, default value and description
- Makes your code reusable
- Values can be passed using a `.tfvars` file or CLI

outputs.tf

- Shows useful information **after** deployment.
- Displays values such as instance ID, public IP, etc.

```
output "instance_id" {
  value     = aws_instance.web.id
  description = "ID of the EC2 instance"
}
```

KEY POINTS

- Outputs important values
- Helps you verify what was created
- Useful for automation and other tools
- Can be referenced by other modules

providers.tf

- Configures the cloud provider (e.g. AWS, Azure, GCP).
- Tells Terraform which provider to use and its settings.

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

KEY POINTS

- Specifies required providers and versions
- Configures provider settings (region, credentials, etc.)
- Can include multiple providers

terraform.tfvars

- Provides values for your **variables**.
- Keeps **sensitive** or **environment-specific values** out of your main code.

```
instance_type = "t2.micro"
ami_id       = "ami-1234567890abcdef"
region      = "us-east-1"
```

KEY POINTS

- Contains actual values
- Works with variables.tf
- Can have different files for different environments (e.g. dev.tfvars, prod.tfvars)
- Should not be committed if it contains sensitive values

IMPORTANT NOTES

- You do not need to use all these files in a small project.
- You can keep everything in `main.tf` for simple setups.
- As your project grows, separate files make your code cleaner and easier to manage.
- Follow a consistent structure to avoid confusion.



JUNIOR ENGINEER TIP

Use a clear and consistent project structure from the beginning. It makes your Terraform code easier to maintain and scale.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



05. HCL SYNTAX EXPLAINED

Page 05/20



WHAT IS HCL?

- **HCL (HashiCorp Configuration Language)** is the language used by Terraform to define infrastructure.
- You write simple, **human readable** configuration files.
- It is designed to be **easy to read**, **consistent** and **maintainable**.

BLOCKS

- Blocks define what you want to configure (provider, resource, variable).
- A block has a **type** and a **label**.

```
resource "aws_instance" "web" {  
  ami          = "ami-1234567890abcdef0"  
  instance_type = "t2.micro"  
}
```

EXPRESSIONS

- Expressions allow you to use variables, functions and combine values.
- They make your configuration dynamic and reusable.

```
resource "aws_instance" "web" {  
  instance_type = var.instance_type  
  tags = {  
    Name = "${var.environment}-server"  
  }  
}
```

COMMON BLOCK TYPES

BLOCK TYPE	PURPOSE
provider	Configure a cloud provider (AWS, Azure, GCP)
resource	Create and manage infrastructure resources
variable	Define input variables
output	Define output values
data	Read existing infrastructure
local	Define local values

EXAMPLE: COMPLETE BLOCK

```
provider "aws" {  
  region = "us-east-1"  
}
```

KEY FEATURES

- Simple and clean syntax.
- Uses **blocks** to define configuration.
- Uses **arguments** to set values.
- Supports **expressions** for dynamic values.
- Allows **comments**.
- Works for all providers (AWS, Azure, GCP and more).

ARGUMENTS

- Arguments are key-value pairs inside a block.
- They set the configuration for that block.

```
resource "aws_instance" "web" {  
  ami          = "ami-1234567890abcdef0"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "web-server"  
  }  
}
```

COMMENTS

- Use comments to explain your configuration.
- Comments are ignored by Terraform.

```
# This is a single line comment  
resource "aws_instance" "web" {  
  # Instance details  
  instance_type = "t2.micro" # Instance type  
  # Multi line numbers  
  # used to explain the purpose  
  tags = {  
    Name = "web-server" # Name tag  
  }  
}
```

BASIC SYNTAX RULES

- 1 Blocks use **{ }** braces.
- 2 Each argument is **key = value**.
- 3 Strings are written in **double quotes "**.
- 4 You can use numbers, booleans (**true/false**), lists and maps.
- 5 File names must end with **.tf** (e.g. main.tf).

IMPORTANT TIPS

- Follow consistent formatting (use terraform fmt).
- Keep your code simple and readable.
- Use comments to explain complex parts.
- Use meaningful names for resources and variables.



JUNIOR ENGINEER TIP

HCL is designed to be simple. Focus on understanding blocks, arguments and expressions. You will learn more advanced features as you build real projects.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



06. PROVIDERS AND RESOURCES

Page 06/20



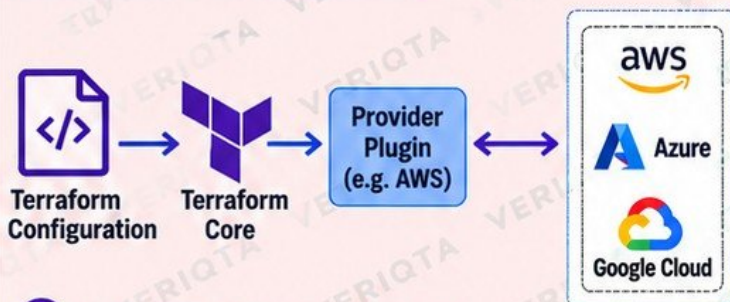
WHAT ARE PROVIDERS?

- Providers are **plugins** that allow Terraform to **communicate** with cloud platforms and other services.
- Each provider exposes **resources**, **data sources** and **features**.
- Examples: AWS, Azure, Google Cloud, Kubernetes, Docker, GitHub, etc.

WHAT ARE RESOURCES?

- Resources are the **infrastructure objects** you want to create, update or delete.
- Each resource belongs to a provider.
- You declare resources in your configuration files using the **resource block**.
- Examples: `aws_instance`, `aws_s3_bucket`, `azurerm_resource_group`, `google_compute_instance`.

HOW PROVIDERS WORK



- 1 Terraform reads your configuration.
- 2 It downloads the required **provider plugin**.
- 3 It uses the provider to call the **platform API**.
- 4 The provider creates, updates or deletes the resources.

PROVIDER BLOCK EXAMPLE

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

KEY POINTS

- `required_providers` specifies the source and version of the provider.
- `provider` block configures the provider settings (such as region, credentials, etc).
- Always use a **version constraint** to maintain stability.

RESOURCE BLOCK EXAMPLE

```
resource "aws_instance" "web" {
  ami           = "ami-1234567890abcdef0"
  instance_type = "t2.micro"
  tags = {
    Name        = "web-server"
    Environment = "dev"
  }
}
```

EXPLANATION

- `resource` declares the **resource type** and a name.
- `aws_instance` is the resource type (from the AWS provider).
- `web` is the **resource name** (you choose this).
- Arguments** inside the block define the configuration for the resource.

RESOURCE NAMING

- The first label is the **resource type** (e.g. `aws_instance`).
- The second label is a **name you define** (e.g. `web`, `db`, `app`).
- Use **meaningful and consistent** names.
- Use lowercase letters, numbers and underscores.
- Avoid spaces and special characters.
- Good names make your code easier to read and maintain.

PROVIDER VERSIONS

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0" # Allows 5.x versions
    }
  }
}
```

VERSION CONSTRAINTS

CONSTRAINT	MEANING
<code>= 5.0.0</code>	Exact version
<code>~> 5.0</code>	Allows 5.x versions
<code>>= 4.0</code>	Version 4.0 or higher
<code><= 5.0</code>	Version 5.0 or lower
<code>> 4.0, < 6.0</code>	Between 4.0 and 6.0
<code>latest</code>	Latest available (not recommended)

COMMON PROVIDERS AND RESOURCE EXAMPLES

PROVIDER	EXAMPLE RESOURCE
AWS	<code>aws_instance</code> , <code>aws_s3_bucket</code>
Azure	<code>azurerm_resource_group</code>
Google Cloud	<code>google_compute_instance</code>
Kubernetes	<code>kubernetes_deployment</code>
Docker	<code>docker_image</code>
GitHub	<code>github_repository</code>



JUNIOR ENGINEER TIP

Always specify provider versions and use clear, meaningful resource names. This makes your infrastructure code more stable, readable and easier to maintain.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



07. YOUR FIRST TERRAFORM PROJECT

Page 07/20



WHAT YOU WILL DO

- Create a **local file** using Terraform.
- Configure **required_providers**.
- Use the **local_file** resource.
- Run the full **Terraform workflow** (init, plan, apply).
- **Verify** the file is created.
- **Understand** each part of the configuration.

TOOLS USED

- Terraform CLI
- **local provider**
(built-in provider for local resources)



Terraform



Creates a local file on your machine

KEY CONCEPTS

- **required_providers** specifies which providers your configuration needs.
- The **local provider** allows Terraform to manage local resources (files, directories, etc.).
- The **local_file** resource creates a file with the specified content.
- You use **blocks**, **arguments** and **HCL syntax** to define it.

1. CREATE PROJECT DIRECTORY

```
mkdir terraform-local-file
cd terraform-local-file
touch main.tf
```

2. CONFIGURE PROVIDER (main.tf)

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}
```

3. ADD RESOURCE (main.tf)

```
resource "local_file" "example" {
  filename = "example.txt"
  content  = "Hello from Terraform!\n"
  file_permission = "0644"
}
```

COMPLETE main.tf FILE

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "example" {
  filename = "example.txt"
  content  = "Hello from Terraform!\n"
  file_permission = "0644"
}
```

4. INITIALIZE TERRAFORM

```
terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/local versions 2.5.0...
- Installing hashicorp/local v2.5.0...
- Installed hashicorp/local v2.5.0

Terraform has been successfully
initialized!
```

5. VALIDATE CONFIGURATION

```
terraform validate
```

Success! The configuration is valid.

6. CREATE EXECUTION PLAN

```
terraform plan
```

Terraform will perform the following actions:
+ create local_file.example

Plan: 1 to add, 0 to change, 0 to destroy.

7. APPLY CONFIGURATION

```
terraform apply
```

Terraform will perform the following actions:
+ create local_file.example

Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Enter a value: **yes**

local_file.example: Creation complete!
Apply complete! Resources: 1 added.

8. VERIFY THE FILE

```
ls -l example.txt
-rw-r--r-- 1 user user 21 Oct 2 12:34 example.txt
```

```
cat example.txt
Hello from Terraform!
```

WHAT HAPPENED?

- Terraform used the **local provider**.
- It created a file named **example.txt** with the content you specified.
- The file is tracked in the **Terraform state**.
- You can change the content and run **terraform apply** again to update it.

CLEAN UP

```
terraform destroy
```

Do you want to perform these actions?
Enter a value: **yes**

Destroy complete! Resources: 1 destroyed.
The file **example.txt** will be removed.



JUNIOR ENGINEER TIP

The local provider is great for learning. It helps you understand how Terraform providers and resources work before using real cloud providers.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



08. TERRAFORM WORKFLOW

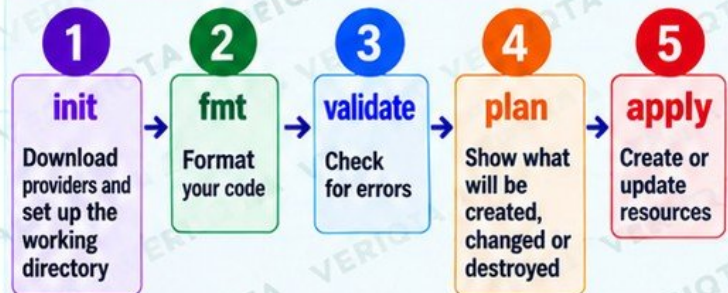
Page 08/20



WHAT IS THE TERRAFORM WORKFLOW?

- A set of CLI commands you run in order to **initialize, format, validate, plan and apply** your configuration.
- It converts your code into real infrastructure.
- Helps you **preview changes** before creating or updating resources.
- Makes your infrastructure changes **safe, predictable and repeatable**.

WORKFLOW OVERVIEW



1. terraform init

- Downloads the required provider **plugins**.
- Initializes the working directory.
- Creates the **.terraform** directory and lock file (**.terraform.lock.hcl**).

terraform init

```
Initializing the backend...
Initializing provider
plugins...
- Finding hashicorp/local
- Installing hashicorp/local
- Installed
Terraform has been
successfully initialized!
```

WHAT IT CHANGES

- Creates **.terraform/** folder.
- Downloads provider plugins.
- Creates/updates **.terraform.lock.hcl**.

2. terraform fmt

- Formats your configuration files.
- Keeps your code clean and consistent.
- Does not change your infrastructure.

terraform fmt

```
main.tf
variables.tf
outputs.tf

Formatted 3 files.
```

WHAT IT CHANGES

- Only changes the formatting of your Terraform files.
- Makes the code easier to read.
- No infrastructure changes.

3. terraform validate

- Checks your configuration for syntax errors.
- Verifies required arguments and values.
- Ensures the configuration is valid.

terraform validate

```
Success! The
configuration is valid.
```

WHAT IT CHANGES

- Does not change any files (unless there are automatic fixes in future versions).
- Does not create, update or delete infrastructure.

4. terraform plan

- Shows what Terraform will do.
- Compares your configuration with the current state.
- Displays resources to add, change or destroy.

terraform plan

```
Terraform will perform
the following actions:
+ create
~ update
- destroy

Plan: 1 to add,
0 to change,
0 to destroy.
```

WHAT IT CHANGES

- Does not change infrastructure.
- Only creates an execution plan.
- Helps you review changes before applying.

5. terraform apply

- Creates, updates or deletes resources based on the plan.
- Asks for confirmation unless you use **-auto-approve**.
- Updates the Terraform state file.

terraform apply

```
Do you want to perform
these actions?
Terraform will perform
the actions described
above.
Only 'yes' will be
accepted to approve.
Enter a value: yes

Apply complete!
Resources: 1 added,
0 changed, 0 destroyed.
```

WHAT IT CHANGES

- Creates, updates or deletes real resources.
- Updates the **terraform.tfstate** file.
- Changes your actual infrastructure.

COMMAND SUMMARY

COMMAND	PURPOSE	CHANGES CONFIG FILES?	CHANGES INFRASTRUCTURE?	KEY OUTPUT
terraform init	Download providers and initialize	No (creates .terraform)	No	.terraform/ and lock file
terraform fmt	Format code	Yes (formatting only)	No	Formatted files
terraform validate	Check for errors	No	No	Validation result
terraform plan	Preview changes	No	No	Execution plan
terraform apply	Create/update resources	No	Yes	Resources created/updated



JUNIOR ENGINEER TIP

Always run **init → fmt → validate → plan → apply**.
This workflow helps you catch errors early and avoid unexpected changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



WHAT IS TERRAFORM STATE?

- Terraform state is a file that keeps track of the **real infrastructure** you have created.
- It maps your configuration to **actual resources** in the cloud or on your local machine.
- It helps Terraform know what resources exist, what has changed and what to **create, update or delete**.
- By default, it is stored in a **local file** (**terraform.tfstate**) in your project directory.

EXAMPLE: terraform.tfstate (SIMPLIFIED)

```
{
  "version": 4,
  "resources": [
    {
      "type": "local_file",
      "name": "example",
      "provider": "provider[\"registry.terraform.io/hashicorp/local\"]",
      "instances": [
        {
          "attributes": {
            "id": "3b1f2c9e4a.txt",
            "filename": "example.txt",
            "content": "Hello from Terraform!",
            "file_permission": "0644"
          }
        }
      ]
    }
  ]
}
```

Resource type
Resource name
Provider
Resource attributes

SENSITIVE INFORMATION RISKS

- State files can contain **sensitive data** such as resource IDs, endpoints, tokens, passwords and other attributes.
- If stored locally and committed to GitHub, this information can be **exposed**.
- Anyone with access to the state file can see sensitive details.
- This can lead to **security breaches**, unauthorized access or infrastructure abuse.

STATE STORAGE OPTIONS

STORAGE TYPE	EXAMPLE	PROS	CONS	USE CASE
Local (default)	terraform.tfstate	Simple, no setup	Not secure, not for teams	Learning and small personal projects
AWS S3	S3 bucket + DynamoDB	Secure, scalable, supports locking	Needs cloud setup and IAM	Production and team environments
Azure Storage	Azure Blob Storage	Secure, scalable, supports locking	Needs cloud setup and IAM	Production and team environments
Google Cloud	GCS bucket	Secure, scalable, supports locking	Needs cloud setup and IAM	Production and team environments



JUNIOR ENGINEER TIP

Always use a remote backend for real projects. It keeps your state file secure, enables collaboration and prevents accidental changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

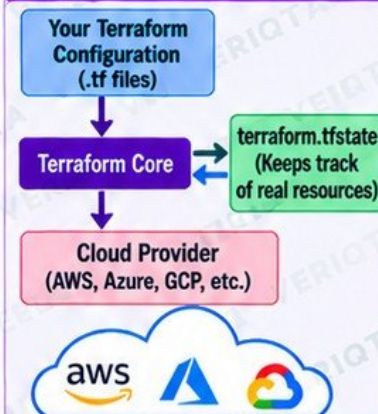
WHAT DOES terraform.tfstate CONTAIN?

- Information about all resources created by Terraform.
- Resource IDs, names, types and configuration values.
- Dependencies between resources.
- Provider information.
- Metadata such as Terraform version and serial number.
- Sensitive data such as resource IDs, endpoints, tokens, passwords and other attributes returned by the provider.

```
{
  "version": 4,
  "terraform_version": "1.6.0",
  "resources": [
    {
      "type": "aws_instance",
      "name": "web",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "attributes": {
            "id": "i-1234567890abcdef0",
            "instance_type": "t2.micro",
            "tags": {
              "Name": "web-server"
            }
          }
        }
      ]
    }
  ]
}
```

Resource type and name
Provider information
Actual resource ID in the cloud
Resource attributes

HOW TERRAFORM USES STATE



WHY STATE MATTERS

- Helps Terraform understand the **current state** of your infrastructure.
- Prevents accidental **duplicate resources**.
- Enables **safe updates**, changes and deletions.
- Tracks **dependencies** between resources.
- Required for accurate **plan** and **apply** operations.

SAFE STATE HANDLING

- Use **remote backends** (e.g. S3 + DynamoDB, Azure Storage, GCS) instead of local state.
- **Restrict access** to the state file using IAM and least privilege.
- Enable **encryption** at rest and in transit.
- Use **state locking** to prevent concurrent changes.
- **Never commit** terraform.tfstate to GitHub.
- Regularly **back up** your state file.

USEFUL COMMANDS

```
# Show current state
terraform show

# List resources in state
terraform state list

# Show a specific resource
terraform state show local_file.example

# Move a resource in state
terraform state mv old_name new_name

# Remove a resource from state
terraform state rm local_file.example
```



10. TERRAFORM VARIABLES

Page 10/20



WHAT ARE TERRAFORM VARIABLES?

- Variables allow you to pass **input values** into your Terraform configuration.
- They make your code **reusable, flexible** and **environment-specific**.
- You can define the **type, default value** and **description** for each variable.
- Values can be provided using **.tfvars files, environment variables, CLI flags** or **defaults**.

COMMON VARIABLE TYPES

TYPE	EXAMPLE
string	"t2.micro"
number	2
bool	true / false
list	["web", "app", "db"]
map	{ env = "dev", region = "us-east-1" }
object	{ name = "web", port = 80 }
set	["a", "b", "c"] (unique values)
tuple	["web", 1, true]

VARIABLE ARGUMENTS

ARGUMENT	PURPOSE
type	Specifies the data type (string, number, etc).
default	Sets a default value (optional).
description	Explains what the variable is for.
nullable	Allows or disallows null values (optional).
sensitive	Hides the value in logs and output (optional).
validation	Adds custom validation rules (optional).

VARIABLE PRECEDENCE (HIGH TO LOW)

- 1 CLI flag: `-var "name=value"`
- 2 CLI flag file: `-var-file="custom.tfvars"`
- 3 Environment variable: `TF_VAR_name`
- 4 **terraform.tfvars** file (auto-loaded)
- 5 ***.auto.tfvars** files (auto-loaded)
- 6 Default value in the variable block

VARIABLE BLOCK SYNTAX

```
variable "instance_type" {  
  type      = string  
  description = "EC2 instance type"  
  default    = "t2.micro"  
}
```

Variable name
Data type
Description
Default value

EXAMPLE: variables.tf

```
variable "instance_type" {  
  type      = string  
  description = "EC2 instance type"  
  default    = "t2.micro"  
}  
  
variable "instance_count" {  
  type      = number  
  description = "Number of instances"  
  default    = 1  
}  
  
variable "tags" {  
  type      = map(string)  
  description = "Common tags"  
  default    = {  
    Environment = "dev"  
    Project     = "veriqta"  
  }  
}
```

EXAMPLE: terraform.tfvars

```
instance_type = "t3.micro"  
instance_count = 2  
tags = {  
  Environment = "prod"  
  Project     = "veriqta"  
}
```

USING VARIABLES IN RESOURCES

```
resource "aws_instance" "web" {  
  instance_type = var.instance_type  
  count         = var.instance_count  
  tags          = var.tags  
}
```

VALIDATION EXAMPLE

```
variable "instance_count" {  
  type      = number  
  description = "Number of instances"  
  validation {  
    condition = var.instance_count > 0 &&  
      var.instance_count <= 10  
    error_message = "Instance count must be  
      between 1 and 10."  
  }  
}
```

SENSITIVE VARIABLE

```
variable "db_password" {  
  type      = string  
  description = "Database  
    password"  
  sensitive = true  
}
```

KEY POINTS

- Use variables to make your code reusable and environment-specific.
- Always set proper **types** and **descriptions**.
- Use **.tfvars files** for different environments (dev, staging, prod).
- Be careful with **sensitive values**.
- Understand **variable precedence** to know which value will be used.
- Use **validation** to prevent invalid values.



JUNIOR ENGINEER TIP

Use variables for anything that may change between environments such as instance types, region, tags and resource names.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



11. OUTPUTS AND LOCAL VALUES

Page 11/20



WHAT ARE OUTPUTS?

- Outputs expose **useful information** about your infrastructure.
- They display values after **terraform apply**.
- You can use outputs to show **resource IDs, IP addresses, URLs** and other important information.
- Outputs make it easy to **access values** without checking the state file.

EXAMPLE: outputs.tf

```
output "instance_id" {
  description = "The ID of the EC2 instance"
  value       = aws_instance.web.id
}
output "instance_public_ip" {
  description = "Public IP address"
  value       = aws_instance.web.public_ip
}
output "instance_url" {
  description = "Application URL"
  value       = "http://${aws_instance.web.public_ip}"
}
```

WHAT ARE LOCAL VALUES?

- Locals allow you to define **reusable values** within your configuration.
- They help **reduce repeated expressions** and keep your code **clean**.
- Locals can be strings, numbers, maps, lists or complex expressions.
- They are useful for **naming, tagging** and **common values** used in multiple resources.

EXAMPLE: locals.tf

```
locals {
  project_name = "veriqta"
  environment  = "dev"
  common_tags = {
    Project      = local.project_name
    Environment  = local.environment
    ManagedBy    = "Terraform"
  }

  instance_name = "${local.project_name}-${local.environment}-web"
}
```

RUNNING AND VIEWING OUTPUTS

- 1 Run **terraform apply**.
- 2 Outputs are displayed after a successful apply.
- 3 You can also view outputs at any time using **terraform output**.
- 4 You can view a specific output using **terraform output <name>**.
- 5 Use **-json** flag to get output in JSON format.

```
$ terraform apply
...
instance_id      = "i0a1b2c3d4e5f6g7h8"
instance_public_ip = "3.22.11.44"
instance_url     = "http://3.22.11.44"
$ terraform output
instance_id      = "i0a1b2c3d4e5f6g7h8"
instance_public_ip = "3.22.11.44"
instance_url     = "http://3.22.11.44"
$ terraform output instance_public_ip
3.22.11.44
$ terraform output -json
{ "instance_public_ip": { "value": "3.22.11.44" } }
```

USING LOCALS IN RESOURCES

```
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  tags          = local.common_tags

  # Using local value for the name
  user_data = <<EOF
    echo "Instance: ${local.instance_name}"
  EOF
}
```

KEY POINTS (OUTPUTS)

- Use outputs to expose **important values**.
- Outputs **do not create** resources.
- You can reference any expression in an output.
- Outputs can be used by **other tools** and CI/CD pipelines.
- Be careful not to expose **sensitive information**.

KEY POINTS (LOCALS)

- Locals **reduce repeated code**.
- They make your configuration **easier to read and maintain**.
- Locals **do not create** resources.
- You can use any valid expression.
- Useful for **naming, tagging** and common values.
- Locals are defined using the **locals block**.

USE CASES

- Outputting resource IDs, IPs, URLs and connection details.
- Sharing values with **other modules or tools**.
- Using locals for **common tags, naming** and environment-specific values.
- Keeping your code **DRY** (Don't Repeat Yourself).

BEST PRACTICES

- Use **meaningful** output names and descriptions.
- Avoid exposing **sensitive data** in outputs.
- Use locals for values used in **multiple places**.
- Keep your outputs focused and relevant.
- Use locals to **simplify complex expressions**.



JUNIOR ENGINEER TIP

Use outputs to get important information after deployment and use locals to keep your configuration clean and maintainable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



12. DATA SOURCES

Page 12/20



WHAT ARE DATA SOURCES?

- Data sources allow you to **read existing infrastructure** instead of creating everything from scratch.
- They get information from your cloud provider or external systems.
- You can use the data in your configuration just like resource attributes.
- They are **read-only** and do not create, update or delete anything.

RESOURCES vs DATA SOURCES

RESOURCES

- Create, update and delete infrastructure.
- Manage the full lifecycle.
- Change cloud resources.
- Have a state entry.
- Example: `aws_instance`, `aws_vpc`, `aws_s3_bucket`.

DATA SOURCES

- Read existing infrastructure.
- Do not create, update or delete.
- Used to look up information.
- Do not change resources.
- Have a state entry (read-only).
- Example: `aws_ami`, `aws_vpc`, `aws_subnet`, `aws_caller_identity`.

HOW DATA SOURCES WORK

- 1 You define a **data block** with a provider and data source type.
- 2 You specify filters or identifiers.
- 3 Terraform **reads the existing** resource from the provider.
- 4 The data is **available** as attributes that you can use in your configuration.

COMMON USE CASES

- Get the latest AMI ID.
- Look up an existing VPC, subnet or security group.
- Read account information (caller identity).
- Get existing DNS records, IAM roles or policies.
- Use existing resources instead of hardcoding values.

COMMON DATA SOURCES

- `aws_ami` – Get AMI information
- `aws_vpc` – Get VPC details
- `aws_subnet` – Get subnet details
- `aws_security_group` – Get security group information
- `aws_caller_identity` – Get current AWS account details
- `aws_route53_zone` – Get Route 53 zone details

EXAMPLE: GET LATEST AMAZON LINUX AMI

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners     = ["amazon"]

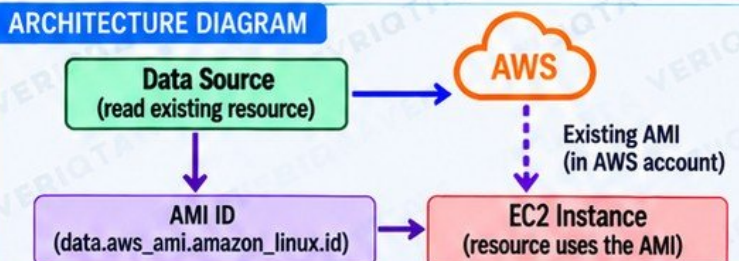
  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

USING DATA SOURCE IN A RESOURCE

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
  tags = {
    Name = "web-server"
  }
}
```

ARCHITECTURE DIAGRAM



EXAMPLE: GET CURRENT AWS ACCOUNT

```
data "aws_caller_identity" "current" {}

# Output the account ID
output "account_id" {
  value = data.aws_caller_identity.current.account_id
}
```

KEY POINTS

- Data sources are **read-only**.
- They help you use existing resources.
- They do **not create** or modify resources.
- You can reference their attributes using the `data.<TYPE>.<NAME>` format.
- They are useful for dynamic and reusable configurations.

BEST PRACTICES

- Use data sources instead of hardcoding values.
- Add filters to get the correct resource.
- Use **data** sources for AMIs, VPCs, subnets and existing resources.
- Combine data sources with variables and locals for flexible configurations.
- Always verify that the data source returns the expected result.



JUNIOR ENGINEER TIP

Use data sources to make your configuration dynamic and reusable. They help you work with existing infrastructure safely.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



13. DEPENDENCIES AND REFERENCES

Page 13/20



WHAT ARE DEPENDENCIES?

- Dependencies define the **order** in which resources are created, updated or deleted.
- Terraform automatically understands dependencies by looking at **references** between resources.
- This ensures resources are created in the **correct order**.
- You can also define explicit dependencies using **depends_on** when needed.

TYPES OF DEPENDENCIES

- 1 **Implicit dependencies** (created automatically through references).
- 2 **Explicit dependencies** (defined using **depends_on**).
- 3 **Data source dependencies** (when data is read and used by a resource).
- 4 **Module dependencies** (when using outputs from other modules).

HOW TERRAFORM DETERMINES EXECUTION ORDER

- 1 Scans your configuration for **references** between resources.
- 2 Builds a dependency graph (DAG - Directed Acyclic Graph).
- 3 Determines the correct order based on the graph.
- 4 Creates resources in the right order.
- 5 If a resource is referenced by another resource, it will always be created first.

IMPLICIT DEPENDENCY (USING REFERENCES)

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "veriqta-vpc"
  }
}

resource "aws_subnet" "public" {
  vpc_id = aws_vpc.main.id # Implicit dependency
  cidr_block = "10.0.1.0/24"
  tags = {
    Name = "public-subnet"
  }
}
```

EXPLICIT DEPENDENCY (depends_on)

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags = {
    Name = "veriqta-igw"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  depends_on = [aws_internet_gateway.igw]
  tags = {
    Name = "public-rt"
  }
}
```

RESOURCE ATTRIBUTES

- Each resource has attributes that you can reference in other resources.
- Common attributes:**
 - **id** - Unique ID of the resource
 - **arn** - Amazon Resource Name
 - **name** - Resource name
 - **vpc_id** - VPC ID
 - **subnet_id** - Subnet ID
 - **public_ip** - Public IP address
 - **private_ip** - Private IP address
 - and many more depending on the resource type.

DEPLOYMENT EXAMPLE WITH DEPENDENCIES

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags = { Name = "veriqta-vpc" }
}

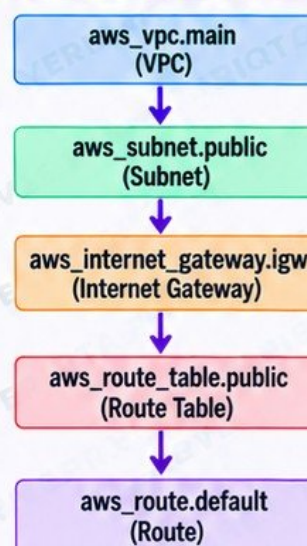
resource "aws_subnet" "public" {
  vpc_id = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
  tags = { Name = "public-subnet" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags = { Name = "veriqta-igw" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  depends_on = [aws_internet_gateway.igw]
  tags = { Name = "public-rt" }
}

resource "aws_route" "default" {
  route_table_id = aws_route_table.public.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.igw.id
}
```

DEPENDENCY GRAPH (EXAMPLE)



KEY POINTS

- Terraform automatically creates **implicit** dependencies when you reference one resource in another.
- Use **depends_on** only when Terraform cannot automatically detect the dependency.
- References ensure the correct **execution order**.
- Dependencies prevent errors caused by missing resources.
- Terraform builds a **dependency graph** and executes resources in the right order.

COMMON USE CASES

- Subnet depends on VPC.
- Internet Gateway depends on VPC.
- Route Table depends on Internet Gateway.
- EC2 instance depends on Subnet and Security Group.
- Security Group depends on VPC.
- RDS depends on Subnet, Security Group and DB Subnet Group.

RESOURCE LIFECYCLE AND DEPENDENCIES

- When a dependency changes, Terraform updates resources in the correct order.
- If a resource is destroyed, dependent resources are also destroyed first.
- This prevents orphaned resources and failed deployments.

BEST PRACTICES

- Use references instead of **depends_on** whenever possible.
- Only use **depends_on** when absolutely necessary.
- Keep your configuration modular and use outputs.
- Always verify the plan to see the execution order.



JUNIOR ENGINEER TIP

Let Terraform handle dependencies automatically by using references. Use **depends_on** only for special cases.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



14. TERRAFORM META-ARGUMENTS

Page 14/20



WHAT ARE META-ARGUMENTS?

- Meta-arguments are special arguments that change how Terraform handles resources and modules.
- They are not specific to any provider.
- The most common meta-arguments are **count**, **for_each** and **lifecycle**.
- They help you manage multiple resources, use dynamic configurations and control resource behaviour.

COMMON META-ARGUMENTS

- count**
Creates multiple similar resources using a number.
- for_each**
Creates multiple resources using a map or set of values.
- lifecycle**
Controls how Terraform manages the resource (create, update, delete).

WHEN TO USE EACH ONE?

META-ARGUMENT	USE CASE
count	Multiple identical resources (number based).
for_each	Multiple resources with different values (map or set).
lifecycle	Control resource behaviour during changes.

1. COUNT META-ARGUMENT

- Creates multiple **identical** resources.
- Uses a number to define how many resources to create.
- Access each resource using **count.index** (starting from 0).

EXAMPLE: CREATE 3 S3 BUCKETS

```
resource "aws_s3_bucket" "example" {
  count = 3

  bucket = "veriqta-bucket-${count.index}"
  tags = {
    Name = "bucket-${count.index}"
  }
}
```

2. FOR_EACH META-ARGUMENT

- Creates multiple resources using a map or set of values.
- Each resource can have different configuration.
- Access each resource using **each.key** and **each.value**.

EXAMPLE: CREATE EC2 INSTANCES

```
resource "aws_instance" "web" {
  for_each = {
    web1 = "t2.micro"
    web2 = "t2.small"
  }

  ami           = "ami-12345678"
  instance_type = each.value
  tags = {
    Name = each.key
  }
}
```

3. LIFECYCLE META-ARGUMENT

- Controls how Terraform handles resource changes.
- Common options: **create_before_destroy**, **prevent_destroy**, **ignore_changes**, **replace_triggered_by**.

EXAMPLE: USE LIFECYCLE OPTIONS

```
resource "aws_instance" "app" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"

  lifecycle {
    create_before_destroy = true
    prevent_destroy       = true
    ignore_changes        = [tags]
  }
  tags = {
    Name = "veriqta-app"
  }
}
```

COUNT vs FOR_EACH

COUNT

- Uses a number.
- Creates identical resources.
- Access using **count.index**.

```
resource "aws_instance" "web" {
  count = 2
  ami   = "ami-123"
}
```

FOR_EACH

- Uses a map or set.
- Creates resources with different values.
- Access using **each.key** and **each.value**.

```
resource "aws_instance" "web" {
  for_each = {
    web1 = "t2.micro"
    web2 = "t2.small"
  }
  ami           = "ami-123"
  instance_type = each.value
}
```

VISUAL COMPARISON

COUNT (3 identical resources)



FOR_EACH (different values)



COMMON LIFECYCLE OPTIONS

OPTION	DESCRIPTION
create_before_destroy	Creates new resource before destroying the old one (zero downtime).
prevent_destroy	Prevents accidental deletion of the resource.
ignore_changes	Ignores changes to specific attributes (e.g. tags).
replace_triggered_by	Forces replacement when a specific attribute changes.

KEY POINTS

- Meta-arguments are used with resources and modules.
- count** is simple but creates identical resources.
- for_each** is flexible and works with maps and sets.
- lifecycle** helps you control how resources are created, updated and deleted.
- Use the right meta-argument based on your use case.

COMMON USE CASES

- Create multiple EC2 instances or S3 buckets.
- Deploy resources for multiple environments.
- Use **for_each** with VPC subnets, security groups or IAM policies.
- Use **lifecycle** to prevent accidental deletion of critical resources.
- Combine meta-arguments with variables and locals for flexible and reusable configurations.



JUNIOR ENGINEER TIP

Use **for_each** when resources need different values and **count** when they are identical. Always use **lifecycle** to protect important resources.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



15. TERRAFORM MODULES

Page 15/20



WHAT ARE TERRAFORM MODULES?

- Modules help you **organize** and **reuse** infrastructure code.
- A module is a collection of Terraform configuration files.
- A **root module** is the main configuration you run.
- **Child modules** are reusable modules called by the root module.
- Modules make your code **cleaner**, **scalable** and **easier to maintain**.

KEY BENEFITS

- 1 Reusable infrastructure code.
- 2 Cleaner and organized configurations.
- 3 Easier maintenance and updates.
- 4 Consistent and standard infrastructure.
- 5 Supports larger and more complex environments.

KEY TERMS

TERM	DESCRIPTION
Root module	The main Terraform configuration you run.
Child module	A reusable module called by the root module.
Module	A collection of .tf files that define infrastructure.
Input variables	Values passed to a module.
Outputs	Values returned by a module.

MODULE DIRECTORY STRUCTURE

```
project-root/
├── main.tf          # Root module
├── variables.tf     # Root input variables
├── outputs.tf      # Root outputs
├── modules/
│   └── vpc/        # Child module
│       ├── main.tf
│       ├── variables.tf
│       ├── outputs.tf
│       └── README.md
```

ROOT MODULE (main.tf)

```
module "vpc" {
  source = "../modules/vpc"
  vpc_name = "veriqta-vpc"
  cidr = "10.0.0.0/16"
}

output "vpc_id" {
  value = module.vpc.vpc_id
}
```

CHILD MODULE (modules/vpc/main.tf)

```
resource "aws_vpc" "this" {
  cidr_block = var.cidr
  tags = {
    Name = var.vpc_name
  }
}
```

CHILD MODULE VARIABLES (modules/vpc/variables.tf)

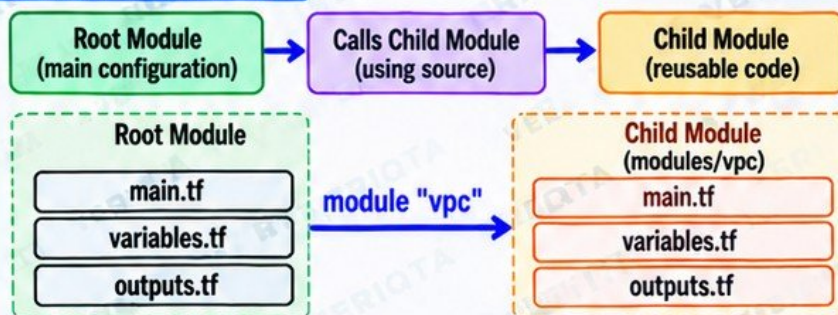
```
variable "vpc_name" {
  type = string
}
variable "cidr" {
  type = string
}
```

CHILD MODULE OUTPUTS (modules/vpc/outputs.tf)

```
output "vpc_id" {
  value = aws_vpc.this.id
}

output "vpc_cidr" {
  value = aws_vpc.this.cidr_block
}
```

HOW MODULES WORK



EXAMPLE: COMPLETE ROOT MODULE

```
module "vpc" {
  source = "../modules/vpc"
  vpc_name = "veriqta-vpc"
  cidr = "10.0.0.0/16"
}

output "vpc_id" {
  value = module.vpc.vpc_id
}
```

EXAMPLE: USING A REMOTE MODULE

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
  name = "veriqta-vpc"
  cidr = "10.0.0.0/16"
  azs = ["us-east-1a", "us-east-1b"]
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  tags = {
    Project = "veriqta"
  }
}
```

KEY POINTS

- The root module calls child modules using the **source** argument.
- Modules use input **variables** and **outputs**.
- You can use local modules (**../modules/...**) or **remote** modules (e.g. Terraform Registry).
- Modules make your infrastructure reusable and consistent.
- Keep modules small, focused and easy to maintain.



JUNIOR ENGINEER TIP

Use modules to keep your code organized and reusable.
Start with small modules and reuse them across different environments.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



16. REMOTE STATE AND BACKENDS

Page 16/20



WHAT IS TERRAFORM STATE?

- Terraform state (**terraform.tfstate**) stores information about the resources you create and manage.
- It maps real infrastructure to your configuration.
- It helps Terraform know what exists, what to create, update or delete.

LOCAL vs REMOTE STATE

LOCAL STATE

- Stored on your local machine.
- Simple to use.
- Not suitable for teams.
- Risk of accidental deletion or corruption.

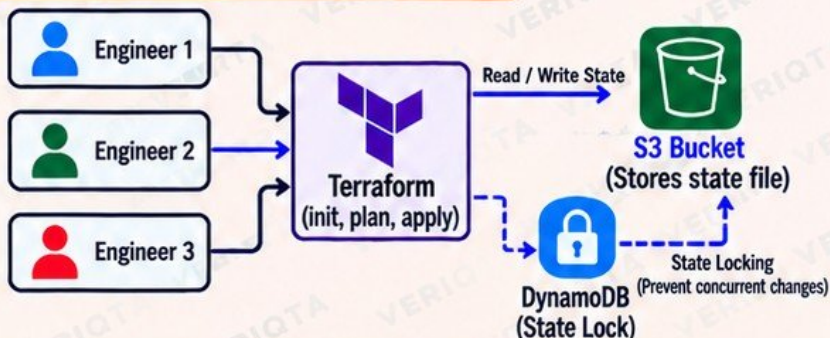
REMOTE STATE

- Stored in a remote backend (e.g. AWS S3).
- Shared by team members.
- Supports locking.
- More secure and reliable.
- Recommended for collaboration.

WHAT IS A BACKEND?

- A backend defines where Terraform stores its state file.
- The backend is configured in the **terraform block**.
- Common backends: AWS S3, Azure Storage, Google Cloud Storage, Terraform Cloud, etc.
- Backends enable team collaboration and state locking.

AWS S3 BACKEND ARCHITECTURE



AWS S3 BACKEND COMPONENTS

- 1 S3 Bucket – stores the state file (**terraform.tfstate**).
- 2 DynamoDB Table – handles **state locking** to prevent concurrent changes.
- 3 IAM Roles/Policies – control access to the state file.
- 4 Versioning – keeps previous state file versions for recovery.
- 5 Encryption – use S3 encryption to protect sensitive data.

EXAMPLE: S3 BACKEND CONFIGURATION

```
terraform {
  backend "s3" {
    bucket = "veriqta-terraform-state"
    key    = "dev/vpc/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    dynamodb_table = "veriqta-terraform-locks"
  }
}
```

INITIALIZE WITH REMOTE BACKEND

```
$ terraform init
Initializing the backend...
Successfully configured
the backend "s3"!

Terraform has been successfully
initialized!
```

DYNAMODB LOCK TABLE (EXAMPLE)

```
resource "aws_dynamodb_table" "tf_lock" {
  name         = "veriqta-terraform-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

S3 BUCKET (EXAMPLE)

```
resource "aws_s3_bucket" "tf_state" {
  bucket = "veriqta-terraform-state"
  force_destroy = false
}

resource "aws_s3_bucket_versioning" "tf_state" {
  bucket = aws_s3_bucket.tf_state.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "tf_state" {
  bucket = aws_s3_bucket.tf_state.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
```

IAM POLICY (EXAMPLE - BASIC)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::veriqta-terraform-state",
        "arn:aws:s3:::veriqta-terraform-state/*"
      ]
    }
  ]
}
```

KEY POINTS

- Use a remote backend for team collaboration.
- S3 stores the state file remotely.
- DynamoDB provides state locking (prevents concurrent changes).
- Use IAM policies to control access.
- Enable versioning and encryption on the S3 bucket.
- Never commit state files to Git.
- Keep sensitive data in the state file protected.
- Remote state improves reliability, security and collaboration.

BEST PRACTICES

- Always use a remote backend for **real** projects.
- Enable S3 versioning and **encryption**.
- Use DynamoDB for state locking.
- Restrict access using **IAM** roles and policies.
- Keep state files out of source control.
- Use separate state files for different environments.
- Monitor and audit access to the state bucket.
- Regularly review permissions and encryption settings.

COMMON USE CASES

- Multiple engineers working on the same infrastructure.
- Preventing state file corruption.
- Storing state securely in AWS S3.
- Using DynamoDB for state locking.
- Managing different environments (dev, staging, prod).



JUNIOR ENGINEER TIP

Use a remote backend (S3 + DynamoDB) from the start to keep your state file secure and avoid conflicts in a team.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



17. TERRAFORM WITH AWS

A simple AWS EC2 provisioning example, provider configuration, credentials, region, and security considerations.

Page 17/20



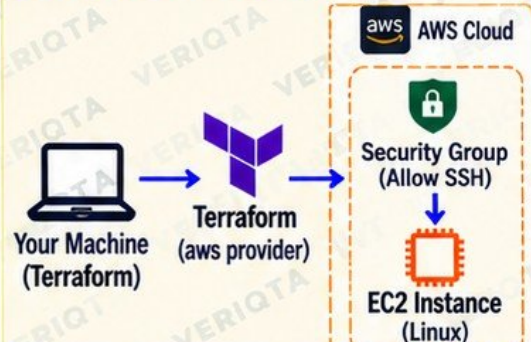
WHAT WE WILL DO?

- Configure the **AWS provider**.
- Set up **credentials** and **region**.
- Create a **security group**.
- Provision a simple **EC2 instance**.
- Use **variables** for common settings.
- Understand key **security considerations**.

PREREQUISITES

- 1 AWS account with IAM user (access key or IAM role).
- 2 Terraform installed (v1.5+ recommended).
- 3 Required AWS permissions to create EC2 and related resources.
- 4 Choose a region (e.g. us-east-1).

ARCHITECTURE DIAGRAM



1. PROVIDER CONFIGURATION (provider.tf)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

2. VARIABLES (variables.tf)

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "ami_id" {
  description = "AMI ID"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}
```

3. SECURITY GROUP (security-group.tf)

```
resource "aws_security_group" "web" {
  name        = "veriqta-sg"
  description = "Allow SSH access"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

4. EC2 INSTANCE (main.tf)

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type
  vpc_security_group_ids = [aws_security_group.web.id]

  tags = {
    Name = "veriqta-ec2"
  }
}
```

5. OUTPUT (outputs.tf)

```
output "instance_id" {
  description = "EC2 Instance ID"
  value       = aws_instance.web.id
}

output "public_ip" {
  description = "Public IP Address"
  value       = aws_instance.web.public_ip
}
```

6. ENVIRONMENT VARIABLES (Credentials)

Use environment variables (recommended):

```
export AWS_ACCESS_KEY_ID="your-access-key"
export AWS_SECRET_ACCESS_KEY="your-secret-key"
export AWS_REGION="us-east-1"
```

Or use an IAM role (recommended for AWS EC2, CI/CD, or production environments).
Avoid hardcoding credentials in your code.

DEPLOYMENT STEPS

- 1 **terraform init** # Download providers
- 2 **terraform fmt** # Format your code
- 3 **terraform validate** # Validate configuration
- 4 **terraform plan** # See what will be created
- 5 **terraform apply** # Create the infrastructure

KEY SECURITY CONSIDERATIONS

- Use IAM roles instead of long-term access keys where possible.
- Follow the principle of least privilege.
- Restrict SSH access to your IP or a specific range.
- Use security groups to control inbound/outbound traffic.
- Keep Terraform state file secure (use remote backend).
- Rotate credentials regularly.
- Avoid exposing sensitive data in code or variables.

COMMON CUSTOMIZATIONS

- Use a different AMI (e.g. Amazon Linux 2, Ubuntu).
- Choose a different instance type.
- Add key pair for SSH access.
- Create the instance in a specific VPC and subnet.
- Add user_data to install software.
- Use tags for better resource management.



JUNIOR ENGINEER TIP

Use variables and outputs to make your configuration reusable.
Never store sensitive information in your code or state file.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



18. TROUBLESHOOTING TERRAFORM

Page 18/20



Provider errors, invalid references, missing variables, authentication failures, state locking, and failed plans.

COMMON TERRAFORM ERRORS

- 1 Provider errors** – issues with provider configuration, version or credentials.
- 2 Invalid references** – referencing resources or variables that do not exist.
- 3 Missing variables** – required variables not provided.
- 4 Authentication failures** – invalid or missing credentials.
- 5 State locking** – state file is locked by another process.
- 6 Failed plans** – errors during terraform plan due to configuration or environment issues.

TROUBLESHOOTING APPROACH

- 1** Read the full error message.
- 2** Understand what Terraform is trying to do.
- 3** Check configuration, variables and credentials.
- 4** Verify provider, region and versions.
- 5** Check the state and locks.
- 6** Use logs and debug mode when needed.
- 7** Make one change at a time and test again.

USEFUL COMMANDS

```
terraform init      # reinitialize
terraform validate  # check syntax
terraform plan      # preview changes
terraform apply     # apply changes
terraform state list # view state
terraform state show <resource> # inspect resource
terraform providers # check providers
terraform version   # check version
TF_LOG=DEBUG terraform plan # detailed logs
```

1. PROVIDER ERRORS

ERROR EXAMPLE

```
Error: Failed to query available provider packages
Could not retrieve the list of available versions for provider hashicorp/aws.
```

CAUSES

- No internet access.
- Wrong provider source or version.
- Network firewall or proxy issues.

FIX

- Check internet connection.
- Verify provider source and version.
- Run `terraform init` again.
- Check proxy/firewall settings.

2. INVALID REFERENCES

ERROR EXAMPLE

```
Error: Reference to undeclared resource
A managed resource "aws_instance.web"
" has not been declared in the root module.
```

CAUSES

- Incorrect resource name.
- Resource not created.
- Typo in reference (e.g. `aws_instance.web.id`).

FIX

- Verify the resource name.
- Check module paths.
- Ensure the resource is defined before referencing it.

3. MISSING VARIABLES

ERROR EXAMPLE

```
Error: No value for required variable
The root module input variable "aws_region"
is not set, and has no default value.
```

CAUSES

- Variable not provided.
- No default value set.
- Variable name mismatch.

FIX

- Provide the variable using `-var` or `.tfvars`.
- Add a default value if appropriate.
- Check the variable name in your code.

4. AUTHENTICATION FAILURES

ERROR EXAMPLE

```
Error: error validating credentials
Error getting credentials from
EC2 Instance Metadata Service:
no EC2 instance role found.
```

CAUSES

- Invalid or missing AWS credentials.
- IAM permissions not sufficient.
- Wrong AWS region or profile.

FIX

- Verify AWS credentials (`aws configure`).
- Check IAM permissions.
- Use the correct profile and region.
- For CI/CD, use IAM roles or environment variables.

5. STATE LOCKING

ERROR EXAMPLE

```
Error: Error acquiring the state lock
Error acquiring the state lock. ConditionalCheckFailedException:
The conditional request failed
```

CAUSES

- Another Terraform process is running.
- Previous run did not release the lock.
- State lock exists in DynamoDB (for S3 backend).

FIX

- Wait for the other process to complete.
- Check DynamoDB lock table and remove stale lock if necessary.
- Use `terraform force-unlock <LOCK_ID>` (if safe).

6. FAILED PLANS

ERROR EXAMPLE

```
Error: creating EC2 Instance: UnauthorizedOperation
you are not authorized to perform
this operation.
```

CAUSES

- Insufficient IAM permissions.
- Invalid configuration.
- Resource limits (e.g. instance quota).
- Region/AZ not available.

FIX

- Check IAM policies and permissions.
- Validate configuration and variable values.
- Check AWS service limits (quotas).
- Verify region and availability zone.

DEBUGGING TIPS

- Use `TF_LOG=DEBUG` for detailed logs.
- Check `.terraform` directory and provider versions.
- Run `terraform validate` to catch syntax errors.
- Use `terraform console` to test expressions.
- Check AWS CLI access (`aws sts get-caller-identity`).

BEST PRACTICES

- Use correct provider versions and sources.
- Keep variables organized and documented.
- Use remote state with locking.
- Validate configuration before apply.
- Use meaningful resource and variable names.
- Follow the principle of least privilege for IAM.

COMMON CHECKLIST

- ✓ Provider configured correctly.
- ✓ Variables provided (or defaults set).
- ✓ Credentials valid and have required permissions.
- ✓ State is not locked (or stale lock removed).
- ✓ Configuration syntax is valid.
- ✓ Region, VPC and resource limits are correct.



JUNIOR ENGINEER TIP

Always read the complete error message. It usually tells you exactly what is wrong and how to fix it.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



19. TERRAFORM SECURITY AND CI/CD

Page 19/20



Secrets protection, version control, code review, automated validation, plan review, and controlled deployment.

WHY SECURITY MATTERS?

- Terraform often manages real, **sensitive infrastructure**.
- Secrets in code can lead to **data breaches**.
- Proper security and CI/CD reduce **human error**.
- Code review and plan review prevent **accidental changes**.
- Automated validation improves **consistency and quality**.
- Controlled deployment keeps your **environments safe**.

KEY SECURITY PRACTICES

- 1 Never store secrets in code or state files.
- 2 Use **environment variables** or a secrets manager (e.g. AWS Secrets Manager).
- 3 **Restrict access** using IAM roles and least privilege.
- 4 Use a **remote backend** with encryption and access control.
- 5 Keep your code in **version control** (Git).
- 6 Use **code review** (Pull Requests) before applying changes.

CI/CD PIPELINE FLOW



1. SECRETS PROTECTION

- Never hardcode secrets in your Terraform code.
- Use **environment variables** or a secrets manager.
- Add sensitive files to **.gitignore**.
- Mark sensitive variables in Terraform.

```
# Use environment variables
variable "db_password" {
  type      = string
  sensitive = true
}

# Access using environment variable
resource "aws_db_instance" "db" {
  password = var.db_password
}
```

2. VERSION CONTROL

- Store Terraform code in Git (GitHub/GitLab).
- Use meaningful commit messages.
- Protect **main** branch.
- Use **Pull Requests** for code review.
- Keep **.tfstate** files out of Git.

```
.gitignore
# Terraform state files
*.tfstate
*.tfstate.*
.terraform/
terraform.tfvars
*.tfvars
.env
# Sensitive files
secrets/
```

3. CODE REVIEW

- Use Pull Requests (PR) for all changes.
- Review code, variables, and resource changes.
- Check for security best practices.
- Ensure plan output is reviewed before apply.

Pull Request: Add VPC module

Conversation 3 | Commits 2 | Checks 5

reviewer1
Looks good. Please confirm the CIDR range.

reviewer2
All security group rules look correct.

All checks have passed
This branch has no conflicts.

4. AUTOMATED VALIDATION

- Use **terraform fmt** to check formatting.
- Use **terraform validate** to check syntax.
- Use **tflint** and **tfsec** for security scanning.
- Integrate these checks in the CI pipeline.

```
# Format code
terraform fmt -check

# Validate configuration
terraform validate

# Run security scan (tfsec)
tfsec .

# Run linter (tflint)
tflint
```

5. PLAN REVIEW

- Always review **terraform plan** output.
- Check what resources will be created, updated or deleted.
- Use PR comments to share plan output in CI/CD.
- Require **approval** before apply.

```
# Generate plan and save to file
terraform plan -out=tfplan

# Show the plan
terraform show tfplan
```

6. CONTROLLED DEPLOYMENT

- Use CI/CD tools (**GitHub Actions**, **GitLab CI**, **Jenkins**, etc.).
- Use separate **environments** (dev, staging, prod).
- Require manual **approval** for production.
- Use least privilege IAM roles.
- Keep **audit logs** of all changes.

```
# Apply the approved plan
terraform apply tfplan
```

KEY SECURITY CONSIDERATIONS

- Use IAM roles instead of long-term access keys.
- Encrypt **remote state** (e.g. S3 with SSE).
- Restrict access to state file using IAM policies.
- Use secrets manager for sensitive data.
- Regularly rotate credentials.

CI/CD TOOLS (EXAMPLES)



BEST PRACTICES

- Keep secrets out of code and state files.
- Use remote **backend** with encryption.
- Automate validation and security scanning.
- Enforce code review and plan review.
- Use **environment-specific** workspaces or separate directories.
- Deploy in a controlled and repeatable way.



JUNIOR ENGINEER TIP

Always review your plan output before applying. Good security and CI/CD practices prevent mistakes and protect your infrastructure.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



20. CAPSTONE: BUILD AND MANAGE INFRASTRUCTURE

Page 20/20

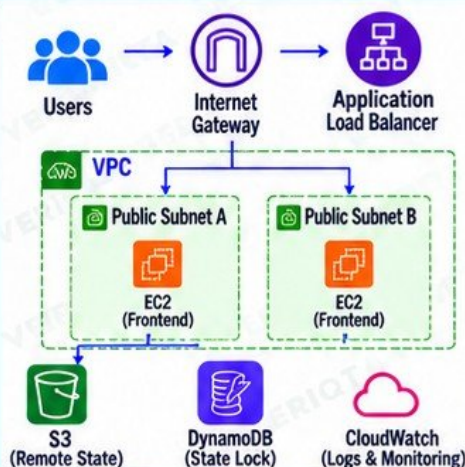


Combine providers, variables, resources, outputs, state, planning, deployment, verification, and safe cleanup.

PROJECT OVERVIEW

- Build a complete, real-world infrastructure using Terraform.
- Use providers, variables, resources, outputs, and remote state.
- Follow a full workflow: plan, deploy, verify and safely clean up.
- Apply best practices for security, organization and maintainability.
- Simulate a production-like environment using AWS.

ARCHITECTURE DIAGRAM



KEY COMPONENTS USED

- 1 Providers – AWS provider configuration.
- 2 Variables – Reusable and flexible configuration.
- 3 Resources – VPC, subnets, security groups, EC2, ALB, etc.
- 4 Outputs – Useful information after deployment.
- 5 Remote State – S3 backend with DynamoDB locking.
- 6 Planning & Deployment – init, plan, apply.
- 7 Verification – Test the infrastructure.
- 8 Safe Cleanup – Destroy resources correctly.

1. PROVIDER CONFIGURATION (provider.tf)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

backend "s3" {
  bucket     = "veriqta-terraform-state"
  key        = "capstone/terraform-tfstate"
  region     = "us-east-1"
  dynamodb_table = "veriqta-terraform-locks"
  encrypt    = true
}

provider "aws" {
  region = var.aws_region
}
```

2. VARIABLES (variables.tf)

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "vpc_cidr" {
  description = "VPC CIDR block"
  type        = string
  default     = "10.0.0.0/16"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "key_name" {
  description = "EC2 key pair name"
  type        = string
}
```

3. MAIN RESOURCES (main.tf)

```
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
  tags = {
    Name = "veriqta-vpc"
  }
}

resource "aws_subnet" "public_a" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "us-east-1a"
  map_public_ip_on_launch = true
  tags = { Name = "public-a" }
}

resource "aws_instance" "web" {
  ami           = "ami-0c82fb55956c7d316"
  instance_type = var.instance_type
  subnet_id    = aws_subnet.public_a.id
  vpc_security_group_ids = [aws_security_group.web.id]
  key_name     = var.key_name
  tags = { Name = "veriqta-web" }
```

4. OUTPUTS (outputs.tf)

```
output "alb_dns_name" {
  description = "Load Balancer DNS"
  value       = aws_lb.app.dns_name
}

output "instance_id" {
  description = "EC2 Instance ID"
  value       = aws_instance.web.id
}

output "vpc_id" {
  description = "VPC ID"
  value       = aws_vpc.main.id
}
```

5. DEPLOYMENT STEPS

- 1 terraform init # Initialize (download providers)
- 2 terraform validate # Check configuration
- 3 terraform plan # Review what will be created
- 4 terraform apply # Deploy infrastructure

6. VERIFICATION

- ✓ Check Terraform output values.
- ✓ Access the application using the ALB DNS.
- ✓ Verify EC2 instances are running.
- ✓ Check resources in AWS console.
- ✓ Confirm state file is stored in S3 and locked in DynamoDB.

7. SAFE CLEANUP

terraform destroy

- Reviews the resources to be destroyed.
- Confirms before deletion.
- Removes all created resources.
- Ensures no resources are left behind.

PROJECT DIRECTORY STRUCTURE

```
terraform-capstone/
├── main.tf           # Main resources
├── provider.tf       # Provider and backend
├── variables.tf      # Input variables
├── outputs.tf        # Outputs
├── versions.tf       # Terraform/AWS versions
├── terraform.tfvars  # Variable values (optional)
└── README.md         # Project documentation
```

BEST PRACTICES

- Use remote state (S3 + DynamoDB).
- Never store secrets in code.
- Use variables and tfvars files.
- Keep code in version control (Git).
- Use code review and pull requests.
- Validate and plan before apply.
- Use least privilege IAM permissions.
- Tag all resources.
- Always clean up resources when done.



JUNIOR ENGINEER TIP

This project brings together everything you have learned. Practice regularly, modify it and make it your own to build real production skills.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta